

CO3 – AT1 COMPUTER VISION EXPERIMENTS

PROGRAMS WITH INPUT AND OUTPUT

1. Preprocessing Impact Analysis

Input and Output

Input: input.jpg. Output: comparison of edge maps before and after denoising/normalization.

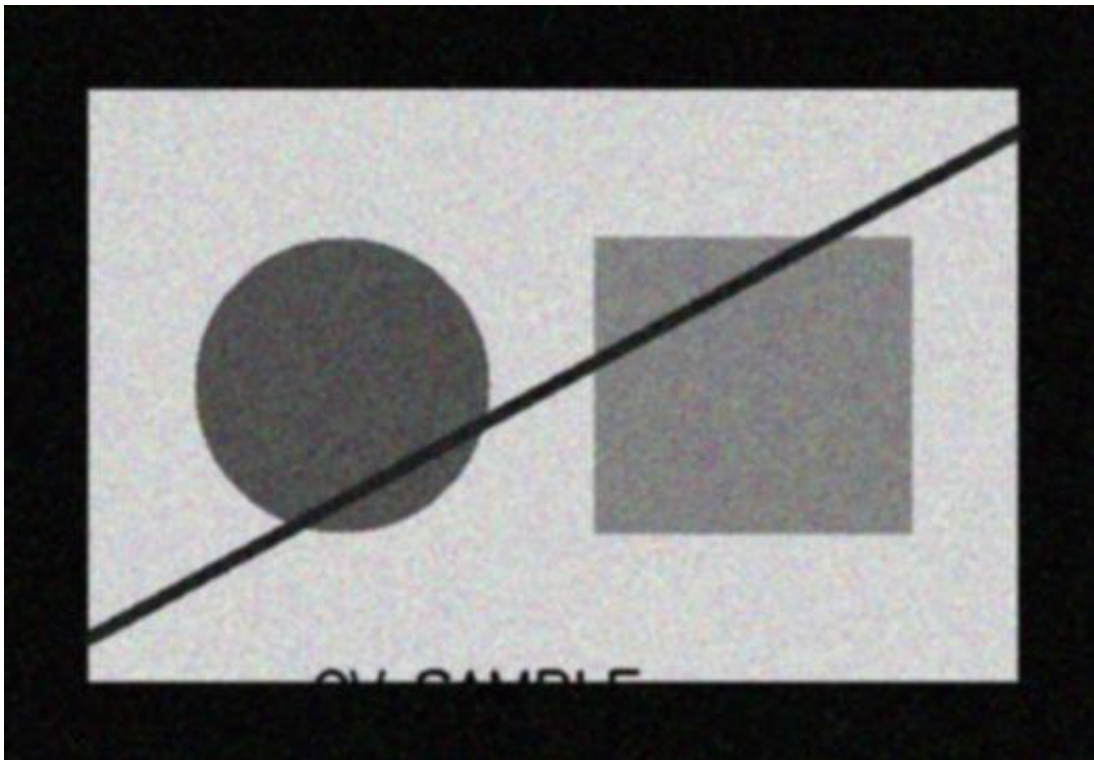
Program:

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
denoised = cv2.GaussianBlur(gray, (5,5), 0)
normalized = cv2.normalize(denoised, None, 0, 255, cv2.NORM_MINMAX)
edges = cv2.Canny(gray, 100, 200)
edges_pre = cv2.Canny(normalized, 100, 200)

plt.figure(figsize=(10,4))
plt.subplot(1,2,1); plt.imshow(edges, cmap="gray"); plt.title("Without preprocessing"); plt.axis("off")
plt.subplot(1,2,2); plt.imshow(edges_pre, cmap="gray"); plt.title("After preprocessing"); plt.axis("off")
plt.show()
```

Sample output image:



2. Image Representation Study

Input and Output

Input: input.jpg. Output: original, grayscale and binary images.

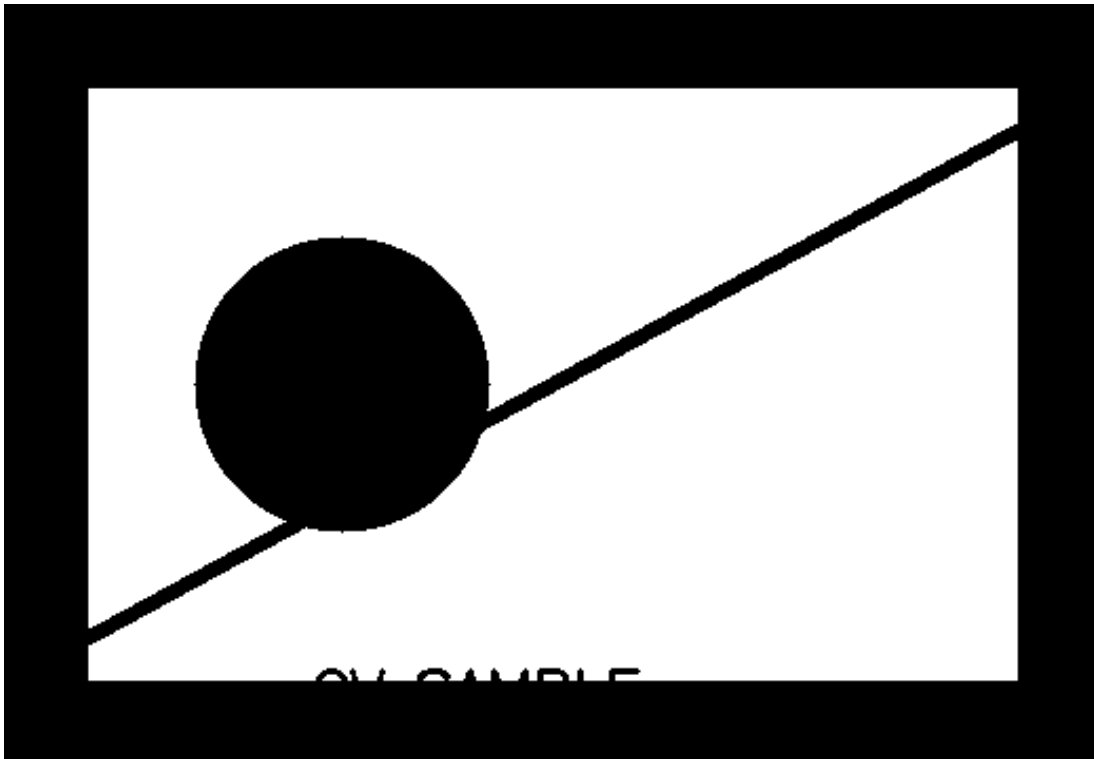
Program:

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

plt.figure(figsize=(12,4))
plt.subplot(1,3,1); plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)); plt.title("Original"); plt.axis("off")
plt.subplot(1,3,2); plt.imshow(gray, cmap="gray"); plt.title("Grayscale"); plt.axis("off")
plt.subplot(1,3,3); plt.imshow(binary, cmap="gray"); plt.title("Binary"); plt.axis("off")
plt.show()
```

Sample output image:



3. Edge Detection Comparison

Input and Output

Input: input.jpg. Output: Sobel and Canny edge images.

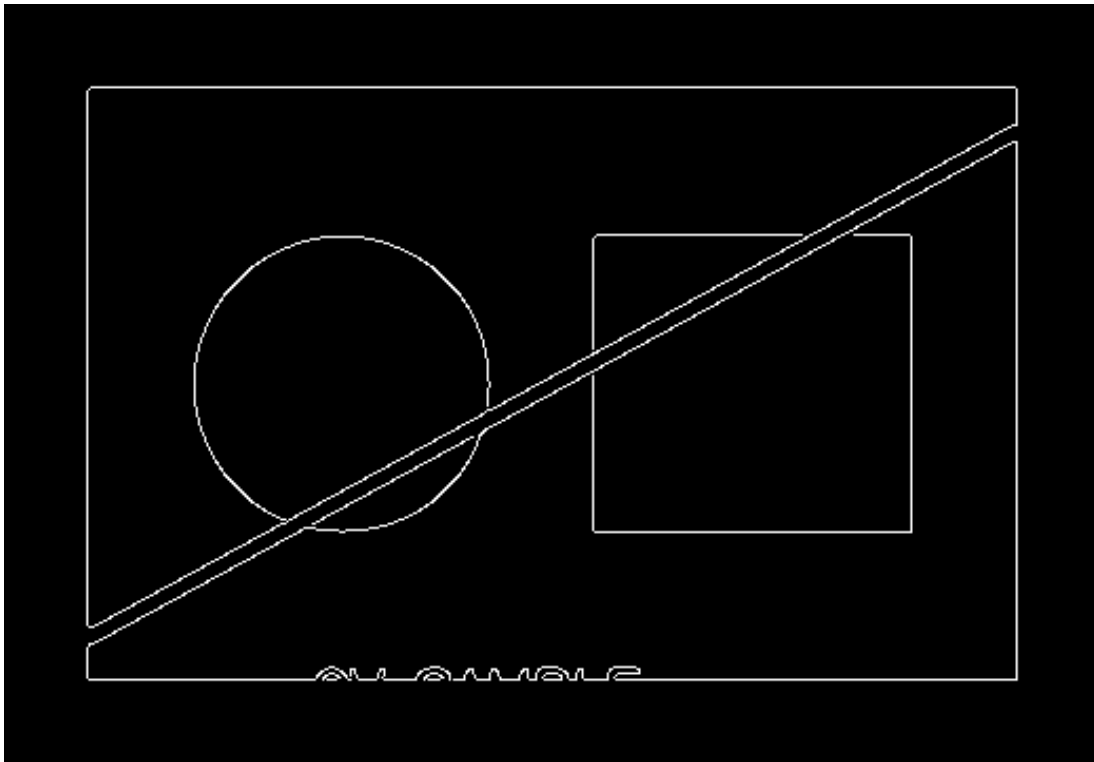
Program:

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg", 0)
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobel = cv2.convertScaleAbs(sobelx)
canny = cv2.Canny(img, 100, 200)

plt.figure(figsize=(10,4))
plt.subplot(1,2,1); plt.imshow(sobel, cmap="gray"); plt.title("Sobel"); plt.axis("off")
plt.subplot(1,2,2); plt.imshow(canny, cmap="gray"); plt.title("Canny"); plt.axis("off")
plt.show()
```

Sample output image:



4. Harris Corner Detection Analysis

Input and Output

Input: input.jpg. Output: image with detected Harris corner points.

Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

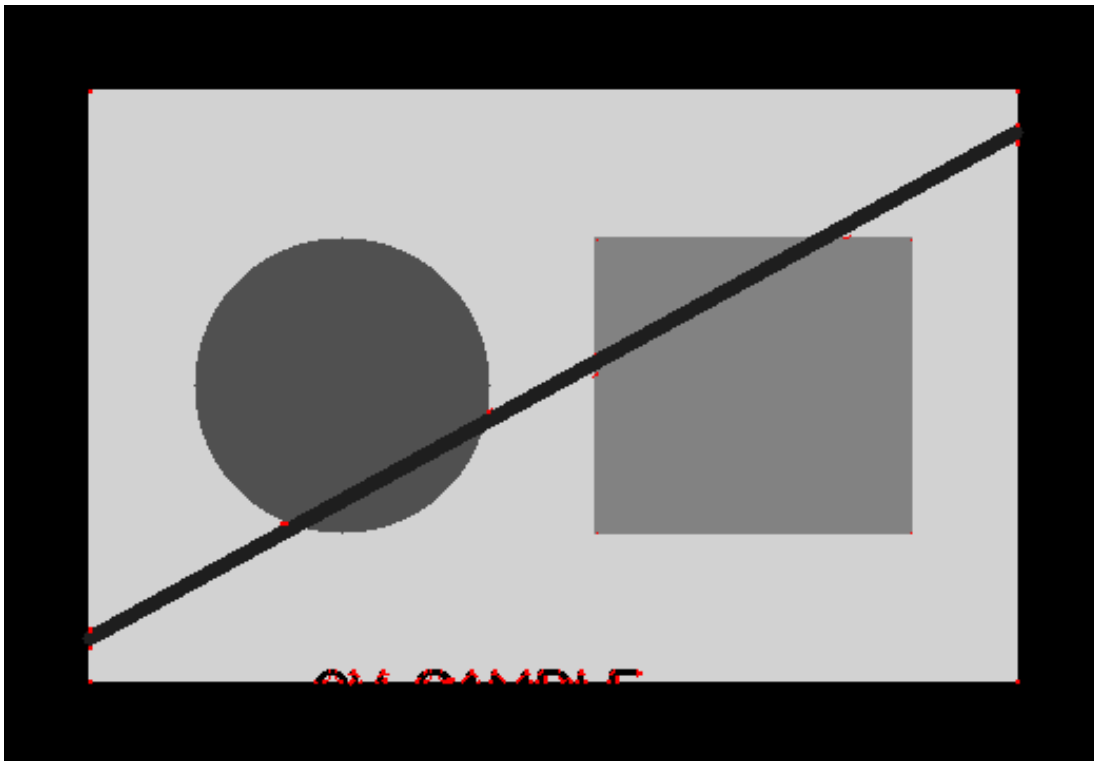
```

img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray32 = np.float32(gray)
dst = cv2.cornerHarris(gray32, 2, 3, 0.04)
result = img.copy()
result[dst > 0.01 * dst.max()] = [0, 0, 255]

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Harris Corners")
plt.axis("off")
plt.show()

```

Sample output image:



5. Effect of Noise on Corner Detection

Input and Output

Input: input.jpg. Output: comparison of Harris corners on original and noisy images.

Program:

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg")
rng = np.random.default_rng(1)
noise = rng.normal(0, 25, img.shape).astype(np.int16)
noisy = np.clip(img.astype(np.int16) + noise, 0, 255).astype(np.uint8)

def harris(im):
    gray = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)
    dst = cv2.cornerHarris(np.float32(gray), 2, 3, 0.04)

```

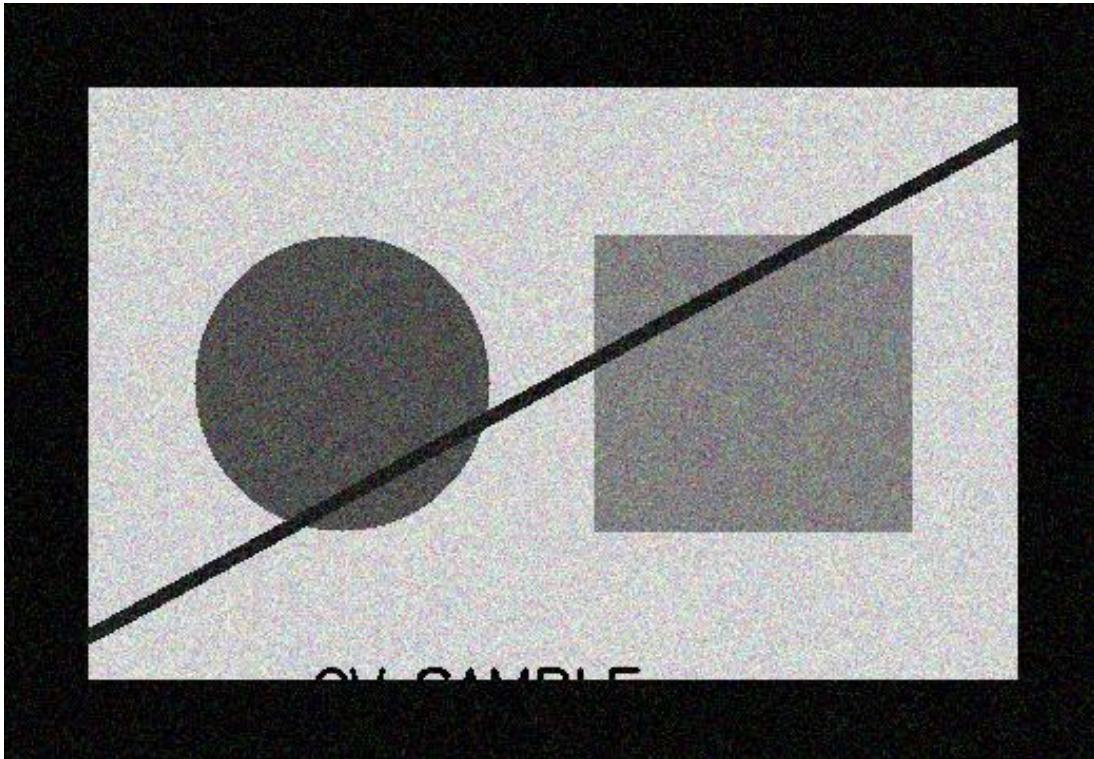
```

out = im.copy()
out[dst > 0.01*dst.max()] = [0,0,255]
return out

plt.figure(figsize=(10,4))
plt.subplot(1,2,1); plt.imshow(cv2.cvtColor(harris(img),cv2.COLOR_BGR2RGB)); plt.title("Original"); plt.axis("off")
plt.subplot(1,2,2); plt.imshow(cv2.cvtColor(harris(noisy),cv2.COLOR_BGR2RGB)); plt.title("Noisy"); plt.axis("off")
plt.show()

```

Sample output image:



6. Hough Transform for Shape Detection

Input and Output

Input: input.jpg. Output: detected straight lines drawn on the image.

Program:

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)
lines = cv2.HoughLinesP(edges, 1, np.pi/180, 50, minLineLength=50, maxLineGap=10)

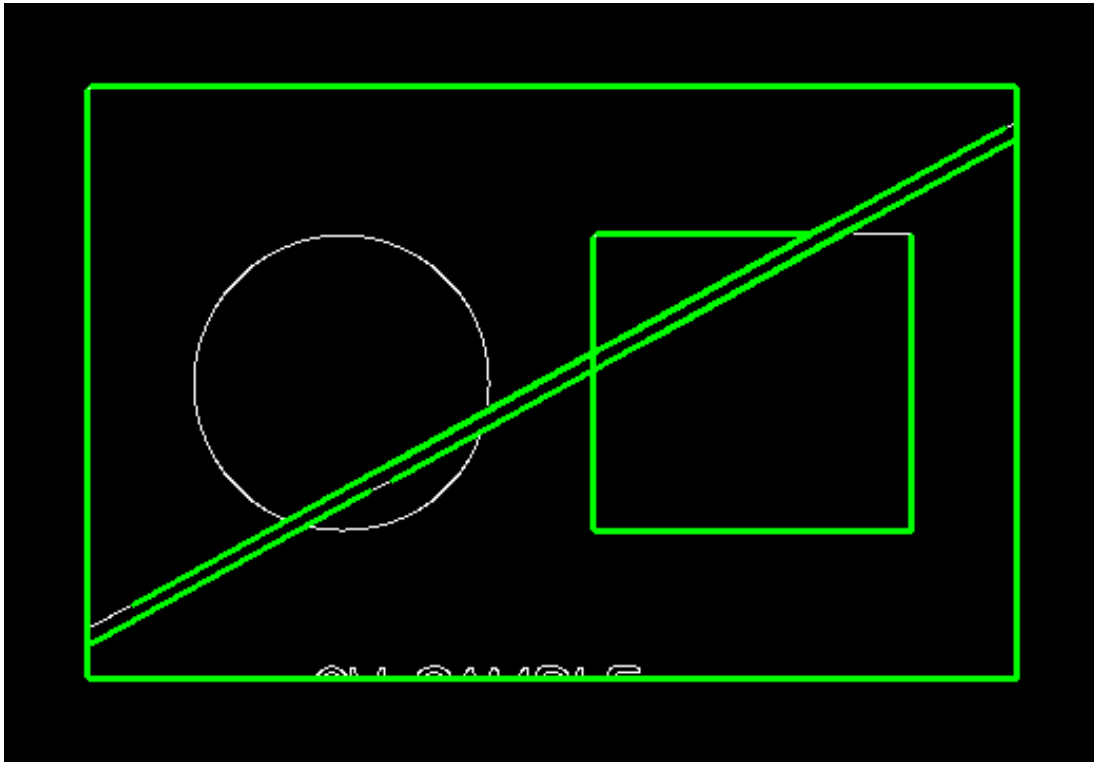
result = img.copy()
if lines is not None:
    for line in lines:
        x1,y1,x2,y2 = line[0]
        cv2.line(result,(x1,y1),(x2,y2),(0,255,0),2)

plt.imshow(cv2.cvtColor(result,cv2.COLOR_BGR2RGB))

```

```
plt.title("Hough Line Detection")
plt.axis("off")
plt.show()
```

Sample output image:



7. Feature Matching using SIFT

Input and Output

Input: image1.jpg and image2.jpg. Output: matched SIFT keypoints and number of good matches.

Program:

```
import cv2
import matplotlib.pyplot as plt

img1 = cv2.imread("image1.jpg", 0)
img2 = cv2.imread("image2.jpg", 0)

sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

bf = cv2.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)
good = [m for m,n in matches if m.distance < 0.7*n.distance]

result = cv2.drawMatches(img1,kp1,img2,kp2,good,None,
                        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
plt.figure(figsize=(12,5))
plt.imshow(result, cmap="gray"); plt.title("SIFT Feature Matches"); plt.axis("off")
plt.show()
print("Good matches:", len(good))
```

Output is primarily numerical/textual (for example, keypoint count, match count, dimensions or ratio).

8. PCA for Feature Reduction

Input and Output

Input: input.jpg. Output: original and reduced feature dimensions plus explained variance.

Program:

```
import cv2
import numpy as np
from sklearn.decomposition import PCA

img = cv2.imread("input.jpg", 0)
pixels = img.reshape(-1,1).astype(np.float32)

pca = PCA(n_components=1)
reduced = pca.fit_transform(pixels)

print("Original feature dimension:", pixels.shape[1])
print("Reduced feature dimension:", reduced.shape[1])
print("Explained variance ratio:", pca.explained_variance_ratio_)
```

Output is primarily numerical/textual (for example, keypoint count, match count, dimensions or ratio).

9. Preprocessing vs Feature Extraction

Input and Output

Input: input.jpg. Output: keypoint counts before and after preprocessing.

Program:

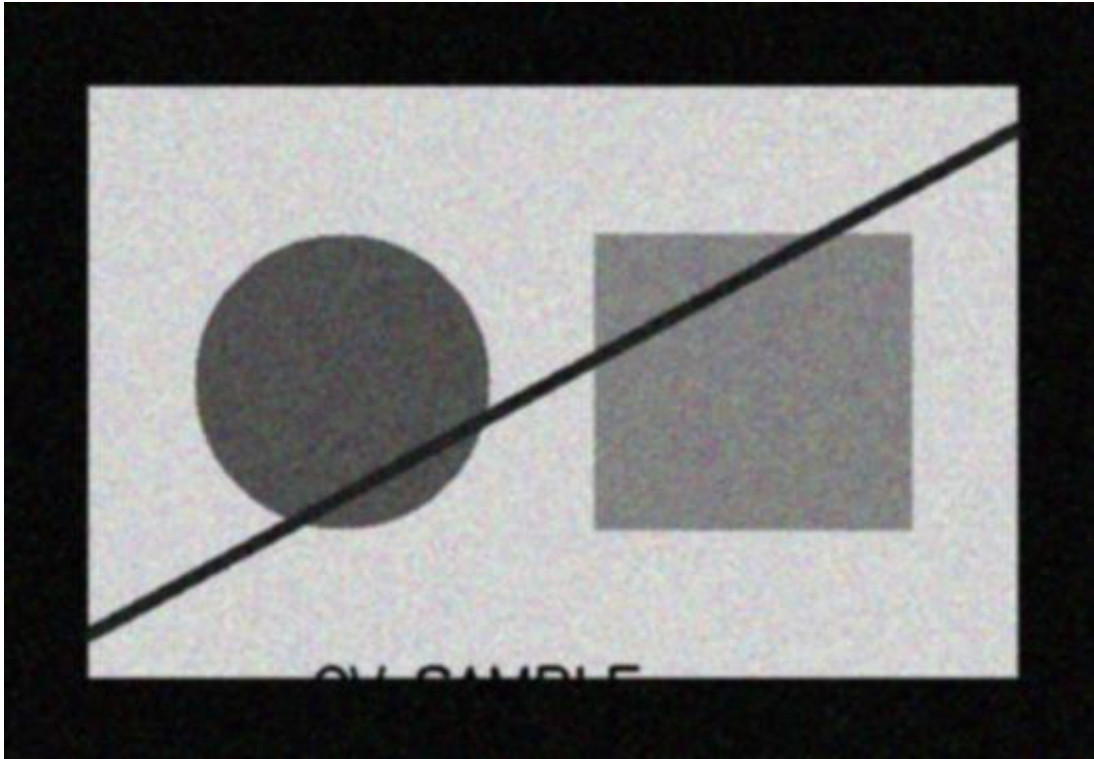
```
import cv2

img = cv2.imread("input.jpg", 0)
sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(img, None)
blur = cv2.GaussianBlur(img, (5,5), 0)
kp2, des2 = sift.detectAndCompute(blur, None)

print("Keypoints without preprocessing:", len(kp1))
print("Keypoints after preprocessing:", len(kp2))
```

Sample output image:



10. Integrated Feature Detection Pipeline

Input and Output

Input: input.jpg. Output: preprocessing, edge detection and feature extraction stages.

Program:

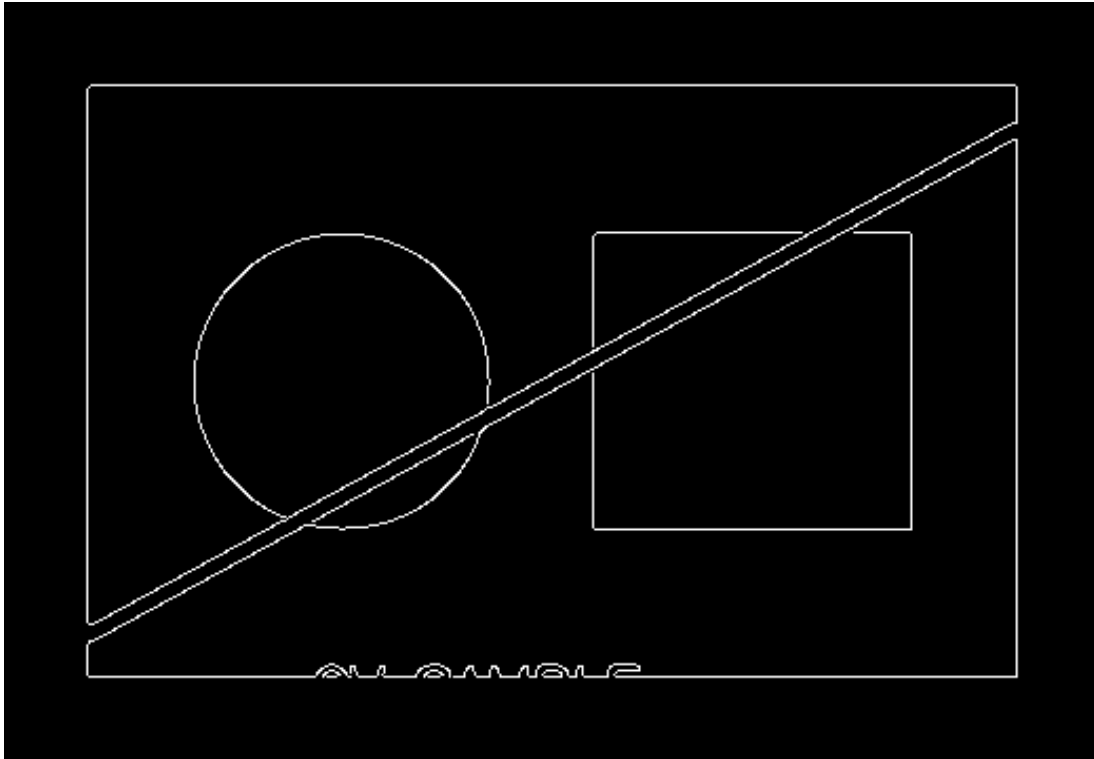
```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
smooth = cv2.GaussianBlur(gray,(5,5),0)
edges = cv2.Canny(smooth,100,200)

sift = cv2.SIFT_create()
kp, des = sift.detectAndCompute(smooth,None)
result = cv2.drawKeypoints(smooth,kp,None,flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

plt.figure(figsize=(12,4))
plt.subplot(1,3,1); plt.imshow(gray,cmap="gray"); plt.title("Preprocessing"); plt.axis("off")
plt.subplot(1,3,2); plt.imshow(edges,cmap="gray"); plt.title("Edges"); plt.axis("off")
plt.subplot(1,3,3); plt.imshow(cv2.cvtColor(result,cv2.COLOR_BGR2RGB)); plt.title("Features"); plt.axis("off")
plt.show()
print("Number of keypoints:", len(kp))
```

Sample output image:



11. Effect of Smoothing on Edge Detection

Input and Output

Input: input.jpg. Output: edge maps before and after smoothing.

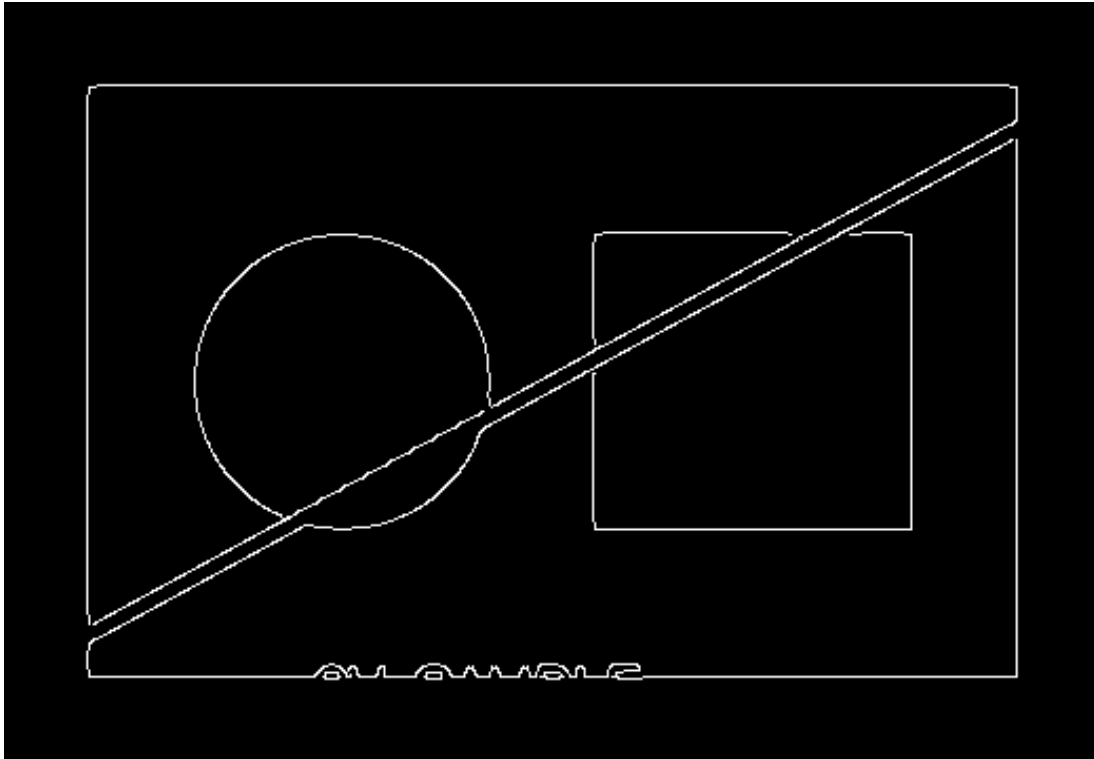
Program:

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg",0)
smooth = cv2.GaussianBlur(img,(7,7),0)
e1 = cv2.Canny(img,100,200)
e2 = cv2.Canny(smooth,100,200)

plt.figure(figsize=(10,4))
plt.subplot(1,2,1); plt.imshow(e1,cmap="gray"); plt.title("Without smoothing"); plt.axis("off")
plt.subplot(1,2,2); plt.imshow(e2,cmap="gray"); plt.title("With smoothing"); plt.axis("off")
plt.show()
```

Sample output image:



12. Gradient-Based Edge Detection Study

Input and Output

Input: input.jpg. Output: gradient magnitude image and maximum gradient value.

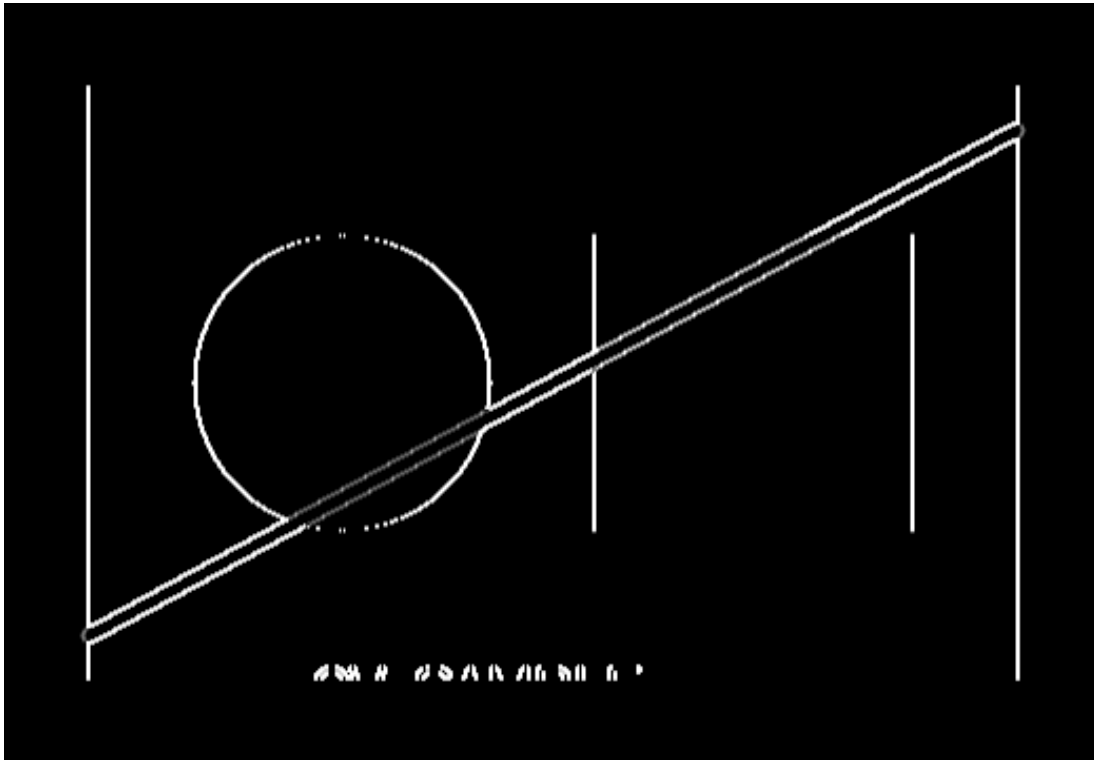
Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg",0)
gx = cv2.Sobel(img,cv2.CV_64F,1,0,ksize=3)
gy = cv2.Sobel(img,cv2.CV_64F,0,1,ksize=3)
magnitude = cv2.magnitude(gx.astype(np.float32), gy.astype(np.float32))
magnitude = cv2.normalize(magnitude,None,0,255,cv2.NORM_MINMAX).astype(np.uint8)

plt.imshow(magnitude,cmap="gray")
plt.title("Gradient Magnitude")
plt.axis("off")
plt.show()
print("Maximum gradient:", magnitude.max())
```

Sample output image:



13. Prewitt vs Sobel Comparison

Input and Output

Input: input.jpg. Output: Prewitt and Sobel edge images.

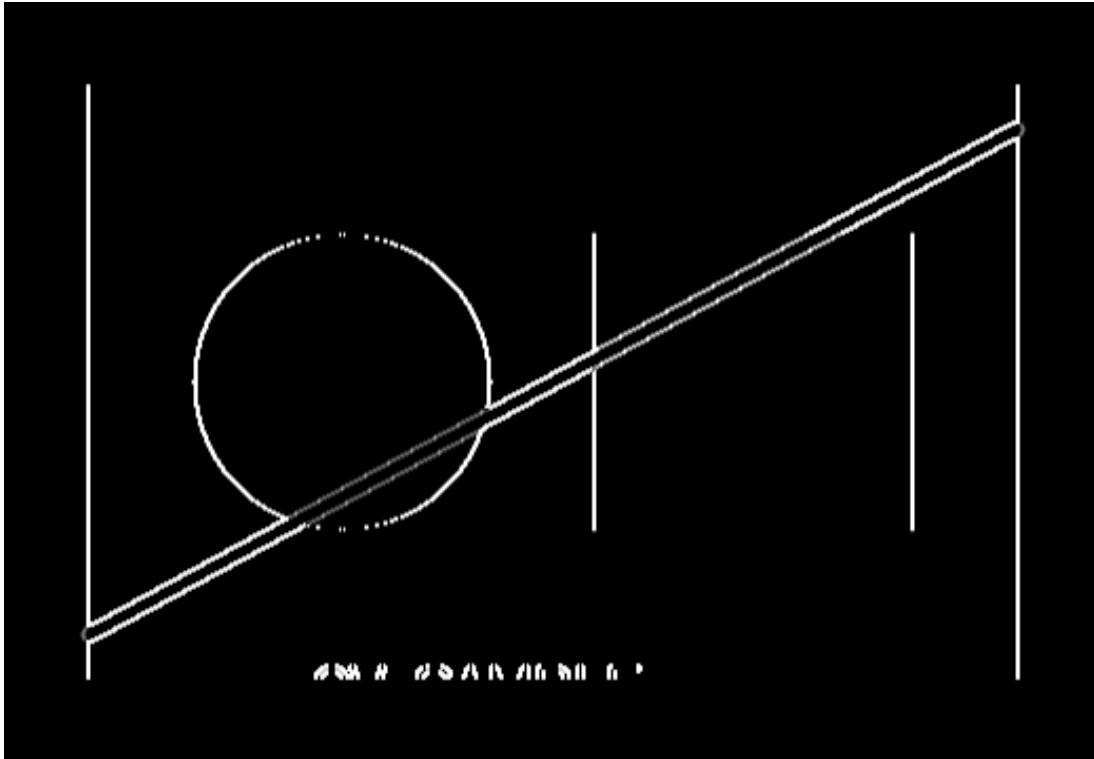
Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg",0)
kx = np.array([[[-1,0,1],[-1,0,1],[-1,0,1]],dtype=np.float32)
prewitt = cv2.filter2D(img,cv2.CV_32F,kx)
prewitt = cv2.convertScaleAbs(prewitt)
sobel = cv2.convertScaleAbs(cv2.Sobel(img,cv2.CV_64F,1,0,ksize=3))

plt.figure(figsize=(10,4))
plt.subplot(1,2,1); plt.imshow(prewitt,cmap="gray"); plt.title("Prewitt"); plt.axis("off")
plt.subplot(1,2,2); plt.imshow(sobel,cmap="gray"); plt.title("Sobel"); plt.axis("off")
plt.show()
```

Sample output image:



14. Effect of Image Scaling on Feature Detection

Input and Output

Input: input.jpg. Output: keypoint count at scales 0.5, 1.0 and 1.5.

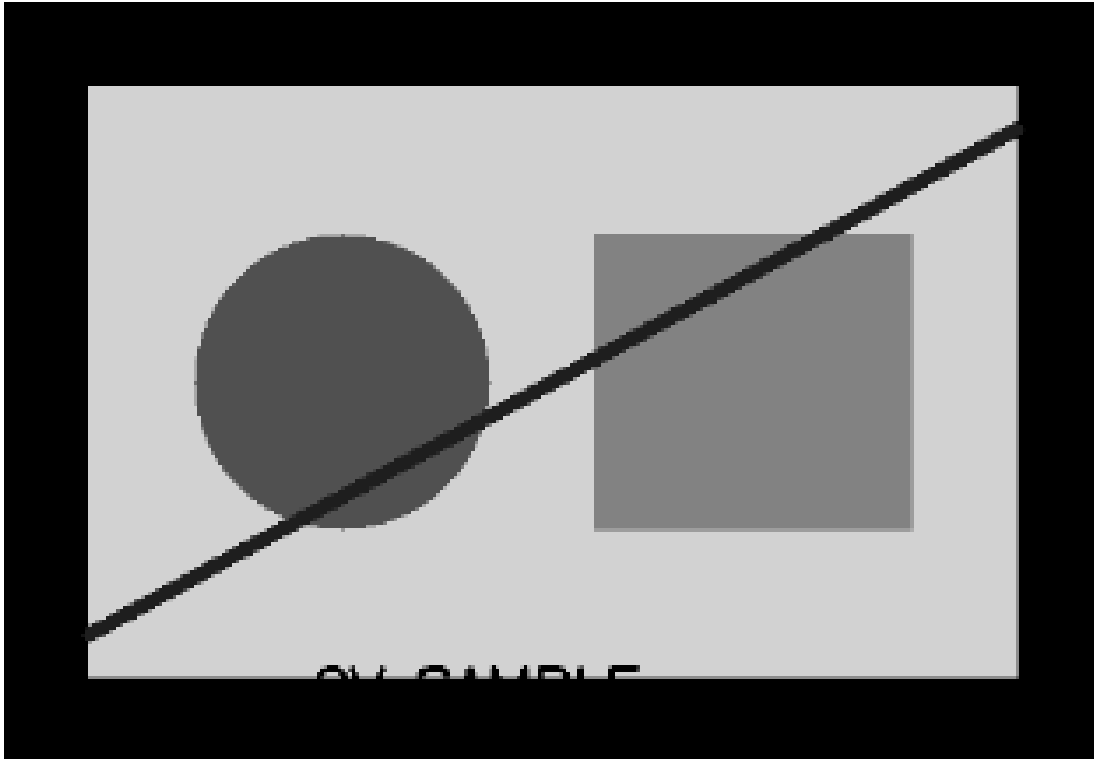
Program:

```
import cv2

img = cv2.imread("input.jpg",0)
sift = cv2.SIFT_create()

for scale in [0.5, 1.0, 1.5]:
    resized = cv2.resize(img,None,fx=scale,fy=scale)
    kp,_ = sift.detectAndCompute(resized,None)
    print(f'Scale {scale}: {len(kp)} keypoints')
```

Sample output image:



15. Rotation Impact on Feature Matching

Input and Output

Input: image1.jpg. Output: good-match count for 0°, 45° and 90° rotations.

Program:

```
import cv2
import numpy as np

img = cv2.imread("image1.jpg",0)
h,w = img.shape
sift = cv2.SIFT_create()
kp1,des1 = sift.detectAndCompute(img,None)
bf = cv2.BFMatcher()

for angle in [0,45,90]:
    M = cv2.getRotationMatrix2D((w//2,h//2),angle,1)
    rot = cv2.warpAffine(img,M,(w,h))
    kp2,des2 = sift.detectAndCompute(rot,None)
    matches = bf.knnMatch(des1,des2,k=2)
    good = [m for m,n in matches if m.distance < 0.7*n.distance]
    print(f"Rotation {angle} degrees: {len(good)} good matches")
```

Output is primarily numerical/textual (for example, keypoint count, match count, dimensions or ratio).

16. Threshold Effect in Edge Detection

Input and Output

Input: input.jpg. Output: Canny edge images for three threshold settings.

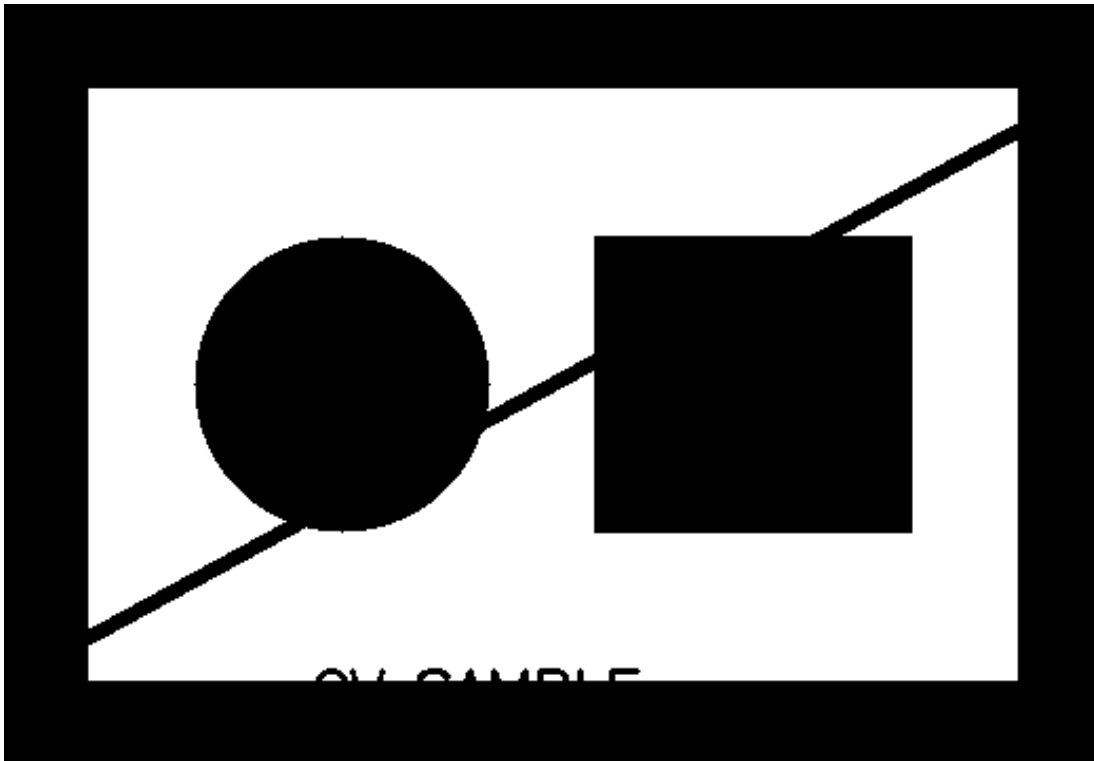
Program:

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg",0)
thresholds = [(50,100),(100,200),(150,250)]

plt.figure(figsize=(12,4))
for i,(a,b) in enumerate(thresholds):
    edges = cv2.Canny(img,a,b)
    plt.subplot(1,3,i+1); plt.imshow(edges,cmap="gray")
    plt.title(f'{a},{b}'); plt.axis("off")
plt.show()
```

Sample output image:



17. Noise Removal Comparison

Input and Output

Input: input.jpg. Output: noisy image and Gaussian, Median and Bilateral filtered images.

Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg")
rng = np.random.default_rng(2)
noise = rng.normal(0,25,img.shape).astype(np.int16)
noisy = np.clip(img.astype(np.int16)+noise,0,255).astype(np.uint8)

gaussian = cv2.GaussianBlur(noisy,(5,5),0)
```

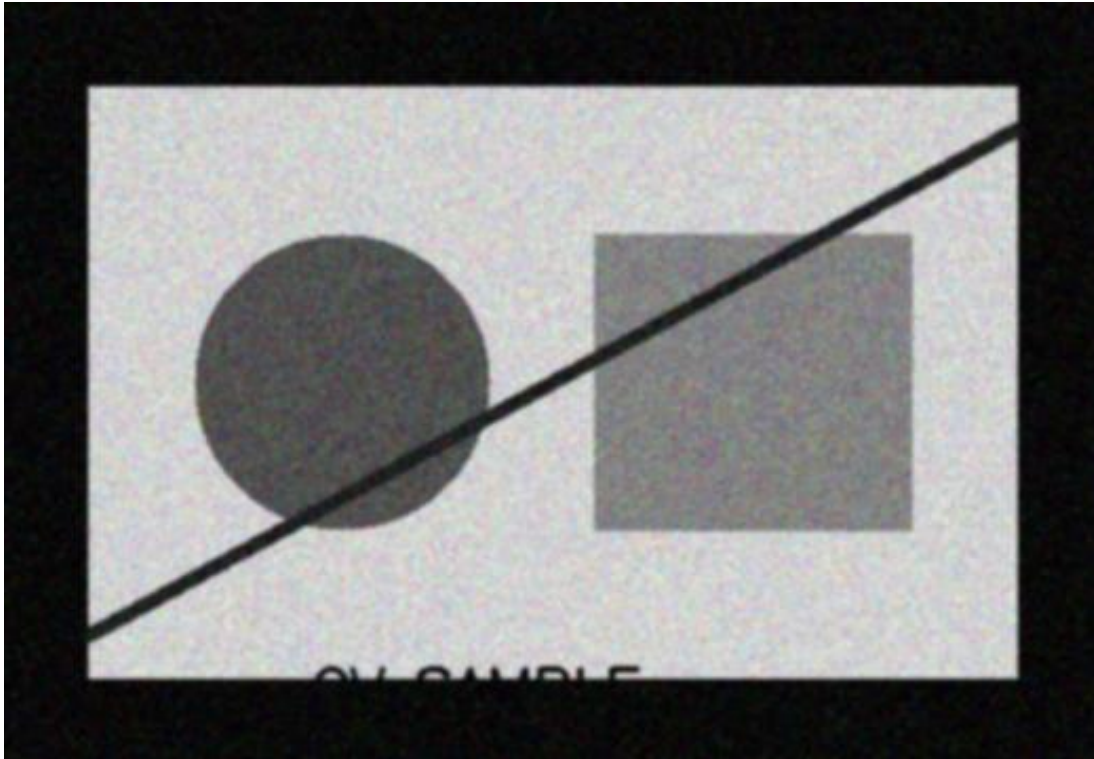
```

median = cv2.medianBlur(noisy,5)
bilateral = cv2.bilateralFilter(noisy,9,75,75)

plt.figure(figsize=(12,4))
for i,(im,title) in enumerate([(noisy,"Noisy"),(gaussian,"Gaussian"),(median,"Median"),(bilateral,"Bilateral")]):
    plt.subplot(1,4,i+1); plt.imshow(cv2.cvtColor(im,cv2.COLOR_BGR2RGB)); plt.title(title); plt.axis("off")
plt.show()

```

Sample output image:



18. Feature Detection in Noisy Images

Input and Output

Input: input.jpg. Output: feature counts for original and noisy images.

Program:

```

import cv2
import numpy as np

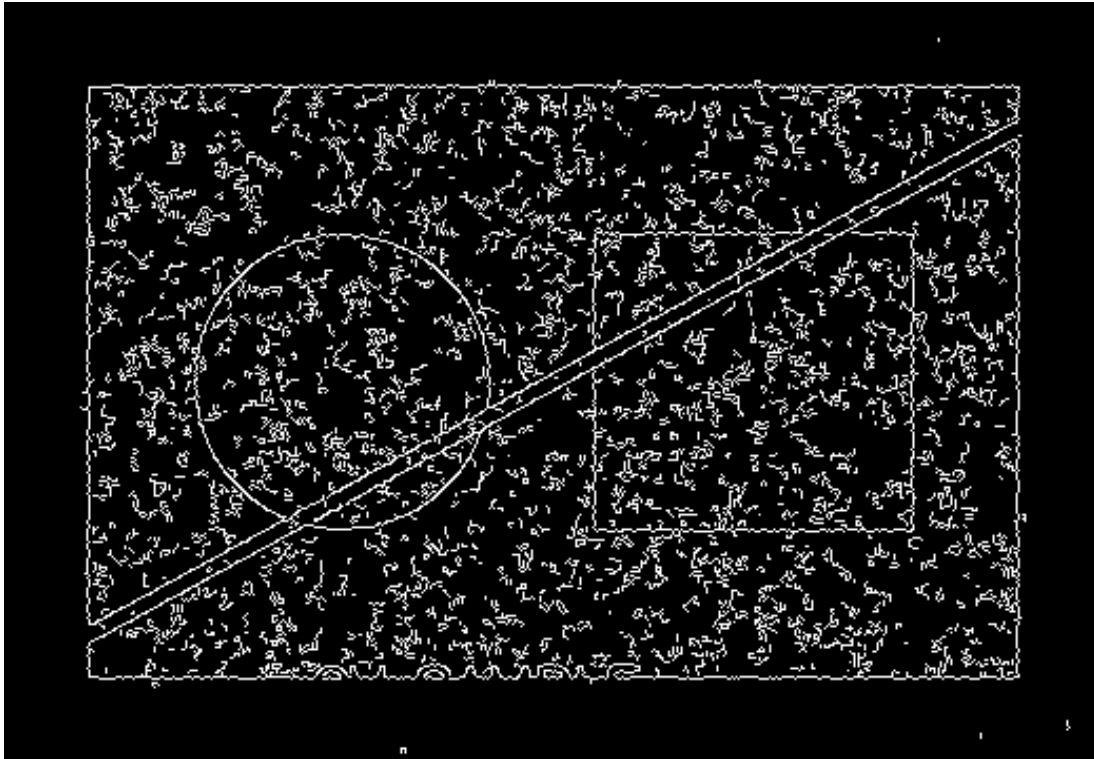
img = cv2.imread("input.jpg",0)
rng = np.random.default_rng(3)
noise = rng.normal(0,25,img.shape).astype(np.int16)
noisy = np.clip(img.astype(np.int16)+noise,0,255).astype(np.uint8)

sift = cv2.SIFT_create()
kp1,_ = sift.detectAndCompute(img,None)
kp2,_ = sift.detectAndCompute(noisy,None)

print("Features in original:", len(kp1))
print("Features in noisy image:", len(kp2))

```

Sample output image:



19. Edge Detection under Illumination Changes

Input and Output

Input: input.jpg. Output: edge-pixel count under darker, normal and brighter illumination.

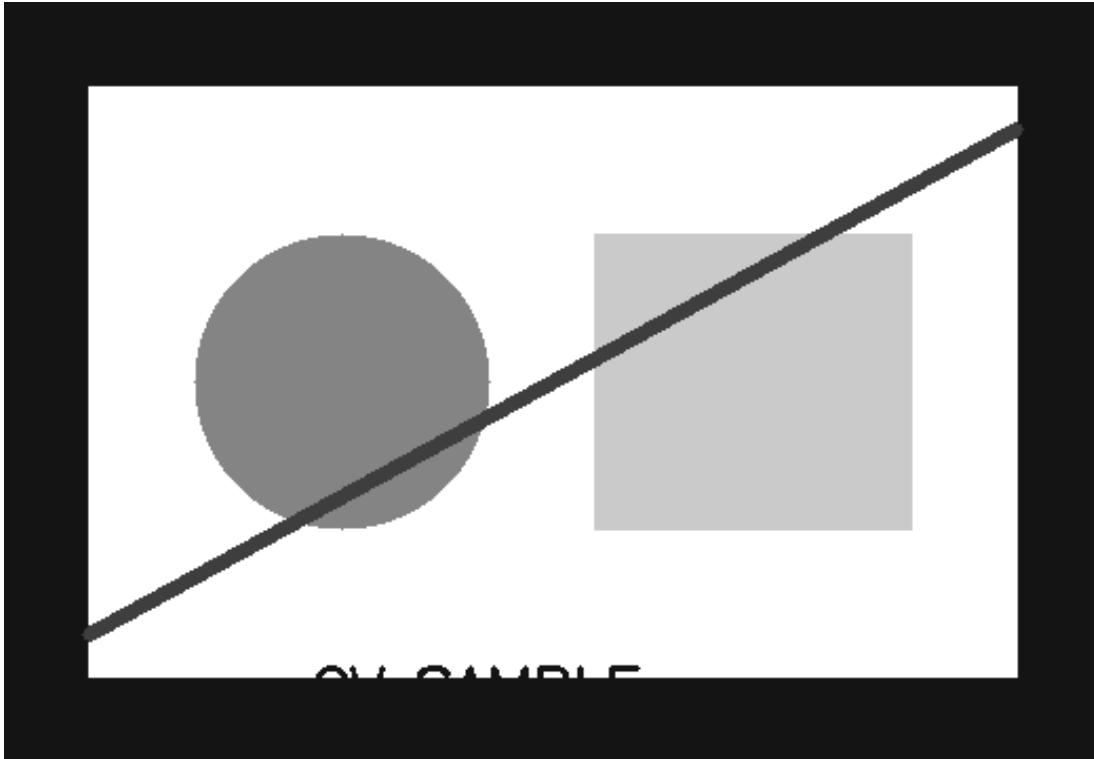
Program:

```
import cv2
import numpy as np

img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)

for beta in [-50,0,50]:
    changed = cv2.convertScaleAbs(gray,alpha=1.0,beta=beta)
    edges = cv2.Canny(changed,100,200)
    print("Illumination offset:", beta, "Edge pixels:", cv2.countNonZero(edges))
```

Sample output image:



20. Corner Detection Parameter Study

Input and Output

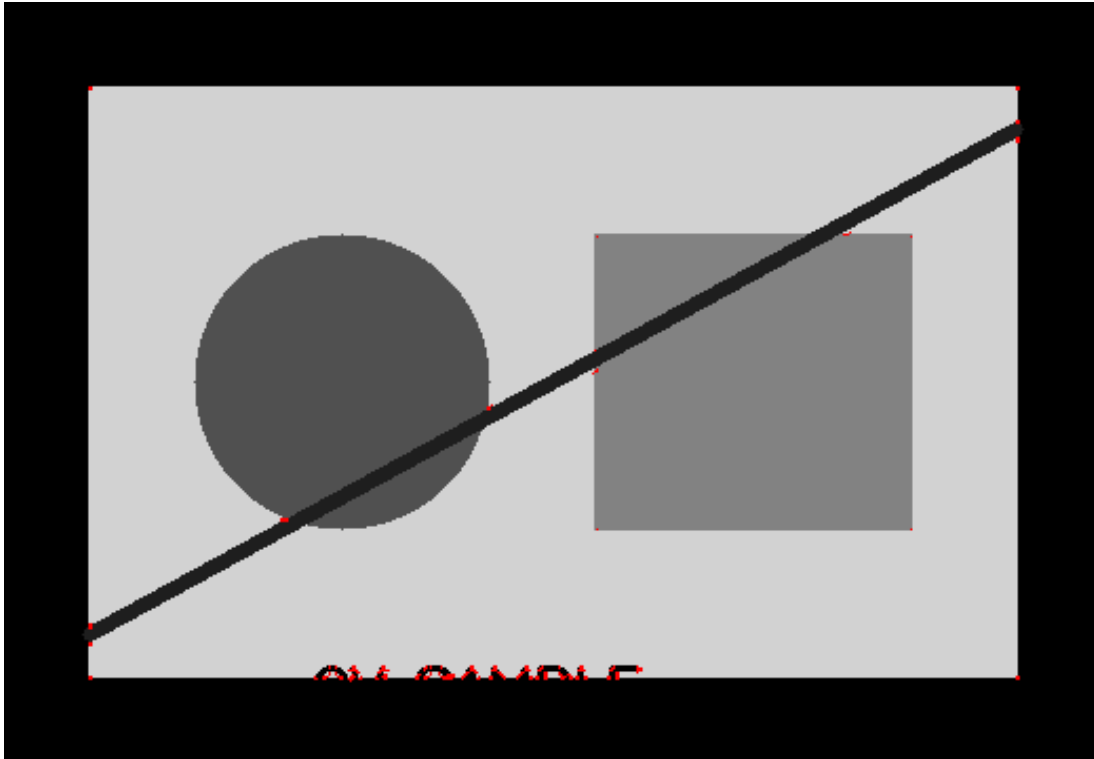
Input: input.jpg. Output: number of detected corners for different Harris parameters.

Program:

```
import cv2
import numpy as np

img = cv2.imread("input.jpg",0)
for blockSize,ksize,k in [(2,3,0.04),(3,3,0.04),(2,5,0.06)]:
    dst = cv2.cornerHarris(np.float32(img),blockSize,ksize,k)
    count = np.sum(dst > 0.01*dst.max())
    print(f"block={blockSize}, ksize={ksize}, k={k} -> corners={count}")
```

Sample output image:



21. Hough Transform Parameter Analysis

Input and Output

Input: input.jpg. Output: number of detected lines for different Hough thresholds.

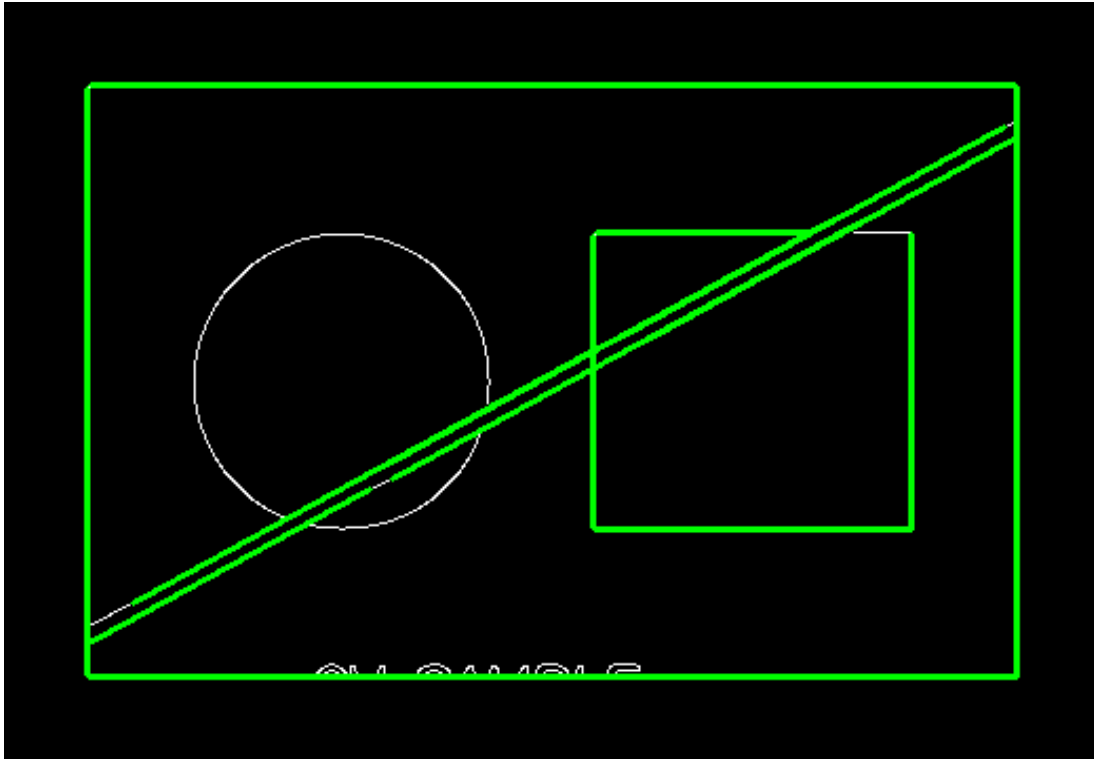
Program:

```
import cv2
import numpy as np

img = cv2.imread("input.jpg",0)
edges = cv2.Canny(img,50,150)

for threshold in [30,50,80]:
    lines = cv2.HoughLinesP(edges,1,np.pi/180,threshold,minLineLength=50,maxLineGap=10)
    count = 0 if lines is None else len(lines)
    print("Hough threshold:", threshold, "Detected lines:", count)
```

Sample output image:



22. Feature Matching Accuracy Evaluation

Input and Output

Input: image1.jpg and image2.jpg. Output: total matches, good matches and matching ratio.

Program:

```
import cv2

img1 = cv2.imread("image1.jpg",0)
img2 = cv2.imread("image2.jpg",0)
sift = cv2.SIFT_create()
kp1,des1 = sift.detectAndCompute(img1,None)
kp2,des2 = sift.detectAndCompute(img2,None)

bf = cv2.BFMatcher()
matches = bf.knnMatch(des1,des2,k=2)
good = [m for m,n in matches if m.distance < 0.7*n.distance]

accuracy = (len(good)/len(matches))*100 if matches else 0
print("Total match pairs:", len(matches))
print("Good matches:", len(good))
print(f"Matching ratio: {accuracy:.2f}%")
```

Output is primarily numerical/textual (for example, keypoint count, match count, dimensions or ratio).

23. PCA Component Selection Study

Input and Output

Input: input.jpg. Output: reduced feature shape and explained variance for selected PCA components.

Program:

```
import cv2
from sklearn.decomposition import PCA
import numpy as np

img = cv2.imread("input.jpg",0)
X = img.reshape(-1,1).astype(np.float32)

for n in [1]:
    pca = PCA(n_components=n)
    Xr = pca.fit_transform(X)
    print("Components:", n)
    print("Reduced shape:", Xr.shape)
    print("Explained variance:", pca.explained_variance_ratio_)
```

Output is primarily numerical/textual (for example, keypoint count, match count, dimensions or ratio).

24. Edge Detection vs Corner Detection

Input and Output

Input: input.jpg. Output: edge map and corner-detection image.

Program:

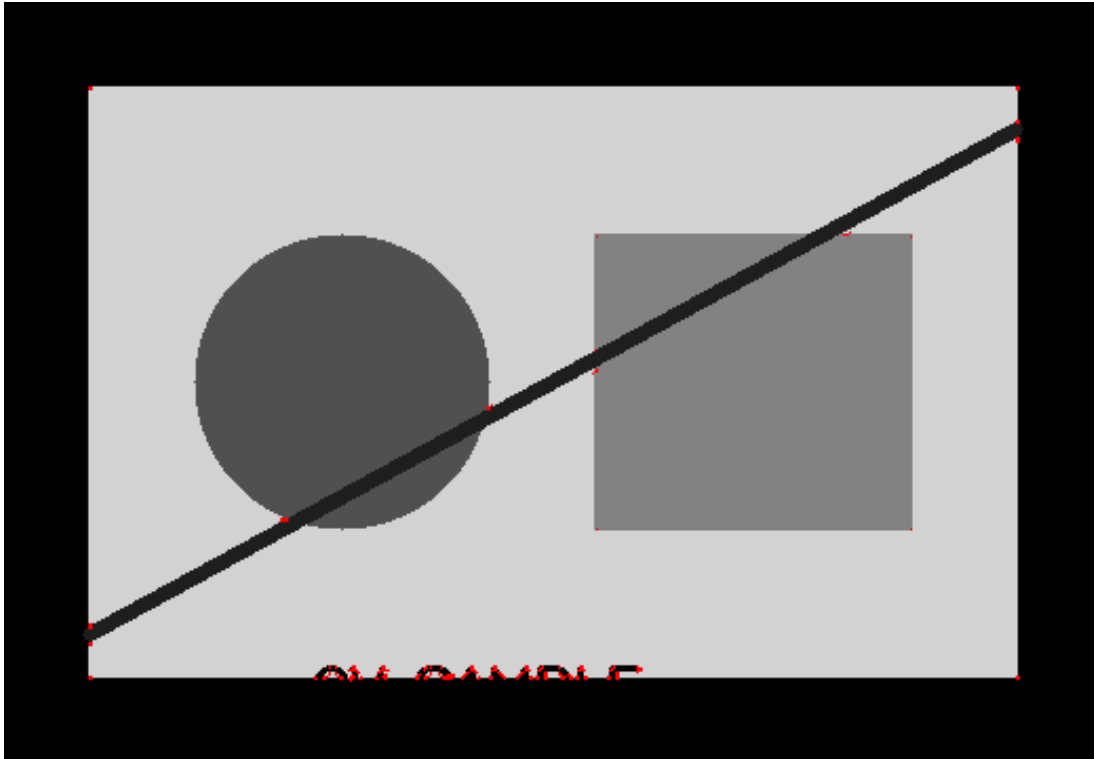
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray,100,200)

dst = cv2.cornerHarris(np.float32(gray),2,3,0.04)
corners = img.copy()
corners[dst > 0.01*dst.max()] = [0,0,255]

plt.figure(figsize=(10,4))
plt.subplot(1,2,1); plt.imshow(edges,cmap="gray"); plt.title("Edges"); plt.axis("off")
plt.subplot(1,2,2); plt.imshow(cv2.cvtColor(corners,cv2.COLOR_BGR2RGB)); plt.title("Corners"); plt.axis("off")
plt.show()
```

Sample output image:



25. Multi-Stage Pipeline Analysis

Input and Output

Input: input.jpg. Output: measurements from preprocessing, edge detection and feature extraction stages.

Program:

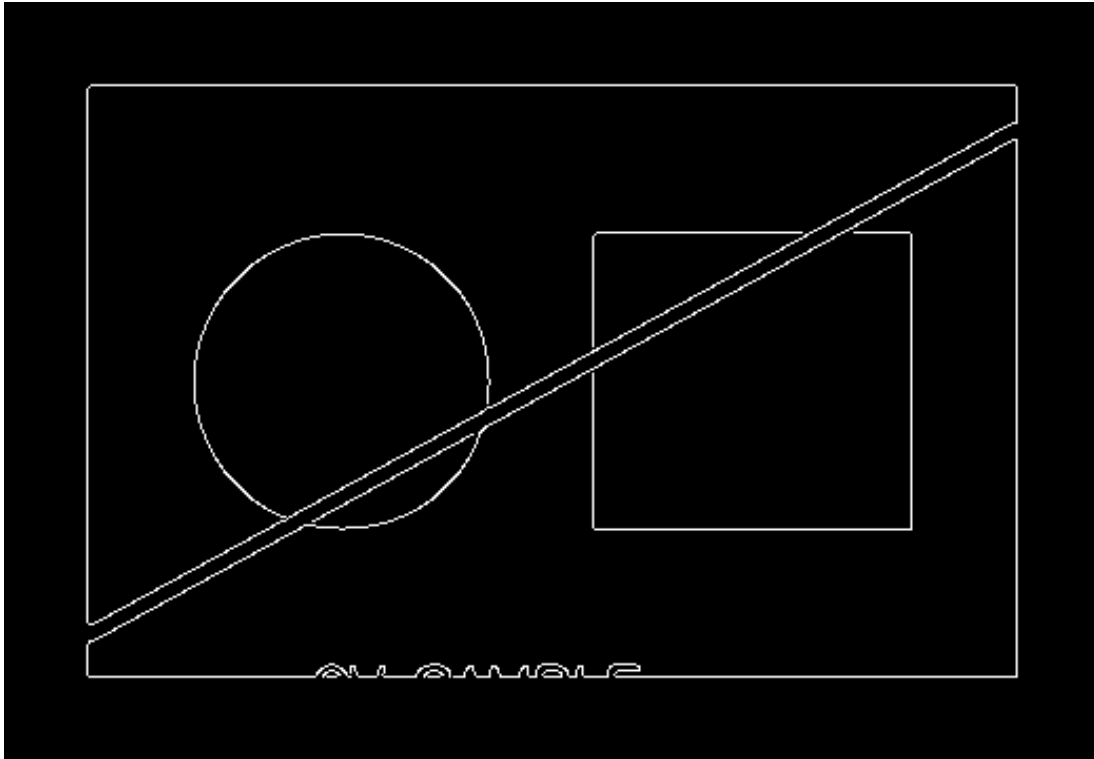
```
import cv2

img = cv2.imread("input.jpg",0)
smooth = cv2.GaussianBlur(img,(5,5),0)
edges = cv2.Canny(smooth,100,200)

sift = cv2.SIFT_create()
kp,des = sift.detectAndCompute(smooth,None)

print("Stage 1 - Input:", img.shape)
print("Stage 2 - Preprocessed:", smooth.shape)
print("Stage 3 - Edge pixels:", cv2.countNonZero(edges))
print("Stage 4 - Feature count:", len(kp))
```

Sample output image:



26. Feature Detection Density Analysis

Input and Output

Input: input.jpg. Output: image size, feature count and feature density.

Program:

```
import cv2

img = cv2.imread("input.jpg",0)
sift = cv2.SIFT_create()
kp,_ = sift.detectAndCompute(img,None)

h,w = img.shape
density = len(kp)/(h*w)

print("Image size:", h, "x", w)
print("Number of features:", len(kp))
print("Feature density:", density)
```

Output is primarily numerical/textual (for example, keypoint count, match count, dimensions or ratio).

27. Effect of Blurring on Feature Detection

Input and Output

Input: input.jpg. Output: keypoint counts after different blur levels.

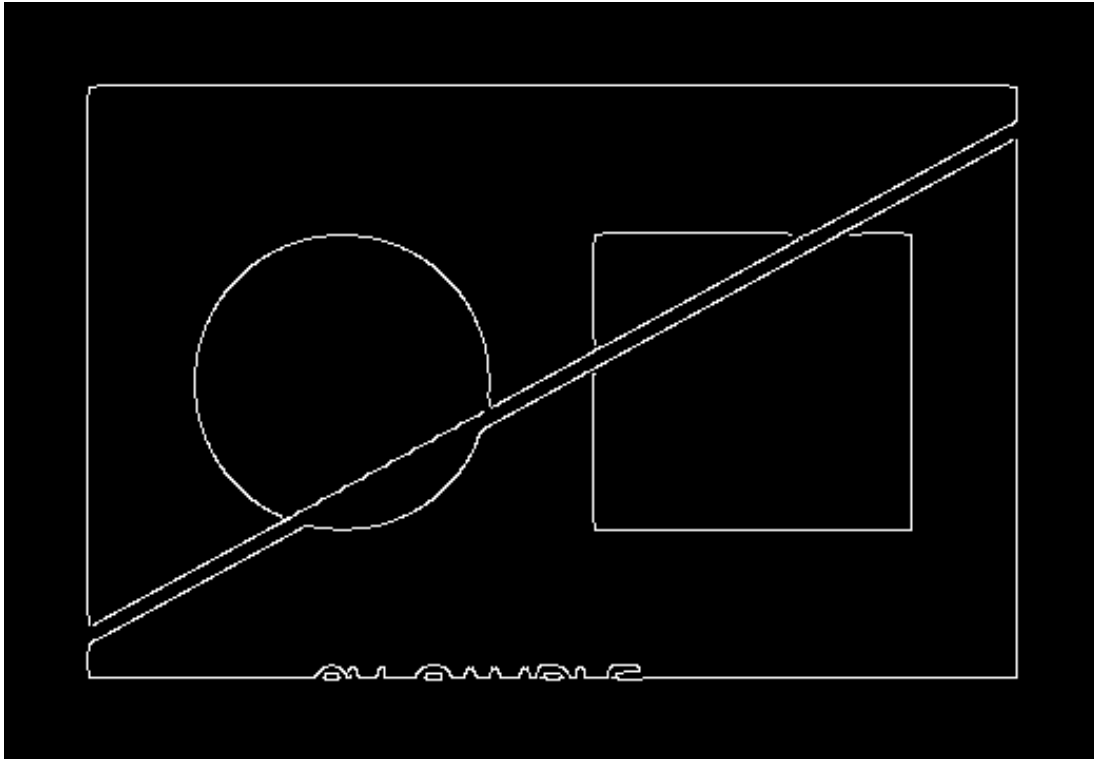
Program:

```
import cv2
```

```
img = cv2.imread("input.jpg",0)
sift = cv2.SIFT_create()

for k in [3,7,11]:
    blurred = cv2.GaussianBlur(img,(k,k),0)
    kp,_ = sift.detectAndCompute(blurred,None)
    print(f"Blur kernel {k}x{k}: {len(kp)} keypoints")
```

Sample output image:



28. PCA Visualization Study

Input and Output

Input: input.jpg. Output: 2D scatter plot of image-patch features after PCA.

Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

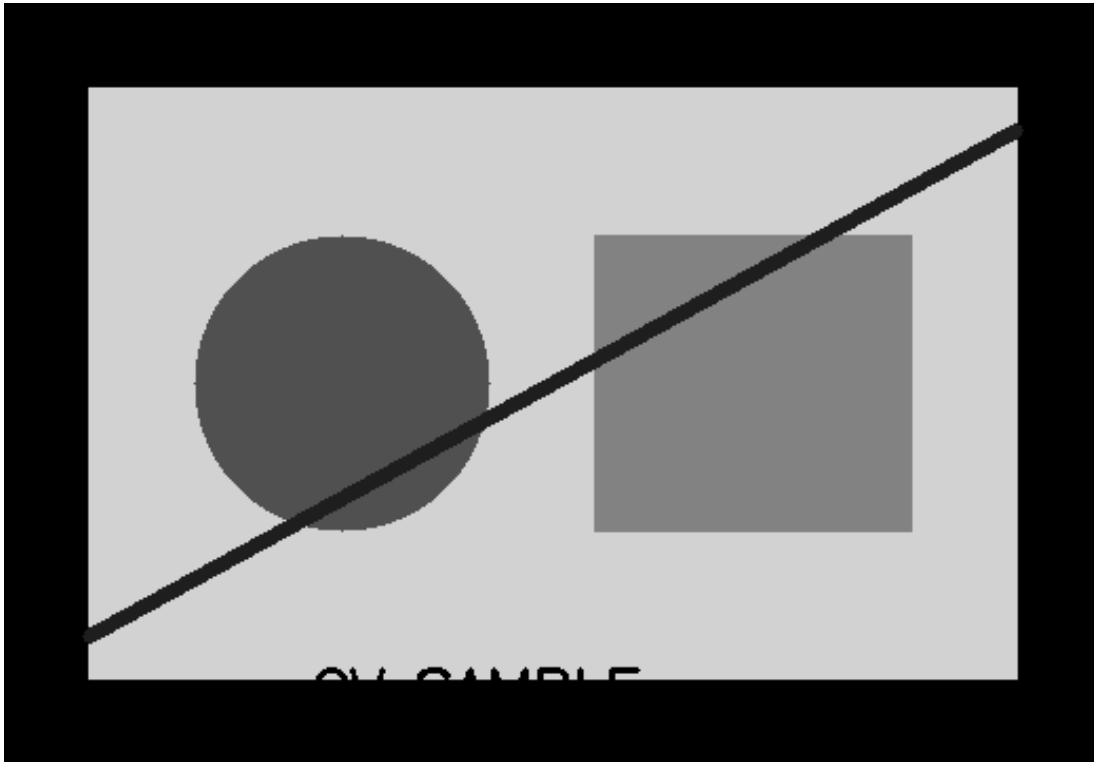
img = cv2.imread("input.jpg",0)
# Divide image into 8x8 patches and represent each patch as a vector
patches = []
for y in range(0, img.shape[0]-8, 8):
    for x in range(0, img.shape[1]-8, 8):
        patches.append(img[y:y+8,x:x+8].flatten())
X = np.array(patches,dtype=np.float32)

pca = PCA(n_components=2)
Z = pca.fit_transform(X)

plt.scatter(Z[:,0],Z[:,1],s=8)
```

```
plt.title("PCA 2D Feature Visualization")
plt.xlabel("PC1"); plt.ylabel("PC2")
plt.show()
```

Sample output image:



29. Feature Matching under Noise and Rotation

Input and Output

Input: image1.jpg. Output: number of good SIFT matches after rotation and noise.

Program:

```
import cv2
import numpy as np

img = cv2.imread("image1.jpg",0)
h,w = img.shape
M = cv2.getRotationMatrix2D((w//2,h//2),30,1)
rot = cv2.warpAffine(img,M,(w,h))

rng = np.random.default_rng(4)
noise = rng.normal(0,20,rot.shape).astype(np.int16)
transformed = np.clip(rot.astype(np.int16)+noise,0,255).astype(np.uint8)

sift = cv2.SIFT_create()
kp1,des1 = sift.detectAndCompute(img,None)
kp2,des2 = sift.detectAndCompute(transformed,None)
bf = cv2.BFMatcher()
matches = bf.knnMatch(des1,des2,k=2)
good = [m for m,n in matches if m.distance < 0.7*n.distance]

print("Good matches after noise + rotation:", len(good))
```

Output is primarily numerical/textual (for example, keypoint count, match count, dimensions or ratio).

30. End-to-End Vision System Experiment

Input and Output

Input: image1.jpg. Output: preprocessing result, detected feature count, descriptor dimensions before/after PCA.

Program:

```
import cv2
import numpy as np
from sklearn.decomposition import PCA

img = cv2.imread("image1.jpg",0)
smooth = cv2.GaussianBlur(img,(5,5),0)

sift = cv2.SIFT_create()
kp1,des1 = sift.detectAndCompute(smooth,None)

# PCA on descriptors
if des1 is not None and len(des1) > 2:
    ncomp = min(20, des1.shape[1], des1.shape[0])
    pca = PCA(n_components=ncomp)
    reduced = pca.fit_transform(des1)
    print("Original descriptor shape:", des1.shape)
    print("PCA descriptor shape:", reduced.shape)

print("Detected features:", len(kp1))
```

Output is primarily numerical/textual (for example, keypoint count, match count, dimensions or ratio).